

A Structural Approach to Program Equivalence in Autoresearch

Abstract

This document formalizes the structural equivalence of markdown programs (`program.md`) using abstract syntax trees, type systems, and feature equivalence. By separating syntactic classification from semantic measurement, we define equivalence classes as purely structural objects. The empirical study then examines how the autoresearch pipeline maps these classes to behavioral outcomes.

1 Parse a `program.md` into a formal object

A `program.md` document has a natural **typed tree structure**. We define a *program term* T inductively:

$$T ::= \text{Doc}(\vec{S}) \mid \text{Sec}(h, \vec{T}) \mid \text{Block}(k, \vec{\ell}) \mid \text{Stmt}(\ell)$$

where h is a heading string, $k \in \{\text{list}, \text{code}, \text{para}\}$ is a block kind, and ℓ is a natural-language sentence. This abstract syntax tree (AST) maps every `program.md` file to a unique term $T \in \mathcal{T}$.

2 Define a structural type system

Rather than comparing raw trees, we type each node. We define a typing judgement $\Gamma \vdash T : \tau$ where the types are drawn from a fixed vocabulary:

$$\tau ::= \text{scope} \mid \text{constraint} \mid \text{strategy} \mid \text{eval} \mid \text{loop} \mid \text{meta} \mid \tau_1 \times \tau_2 \mid \tau^*$$

The typing rules are syntax-directed and deterministic given a small lexicon of heading and keyword patterns. The *type signature* of a program is the multiset of types assigned to its top-level sections:

$$\sigma(p) = \{\{\tau_1, \tau_2, \dots, \tau_k\}\}$$

Two programs with different type signatures are structurally distinct by definition—they belong to different equivalence classes regardless of their content.

3 Equivalence within a type signature

For programs with the same $\sigma(p)$, we extract a feature vector using only syntactic and lexical properties within each typed section:

$$\phi(p) = \bigoplus_{\tau \in \sigma(p)} \phi_\tau(p)$$

where each ϕ_τ is a vector of structural features for sections of type τ :

This feature map $\phi : \mathcal{T} \rightarrow \mathbb{R}^d$ is purely syntactic and computable from the parse tree without semantic reasoning.

Section Type (τ)	Structural Features (ϕ_τ)
scope	# named identifiers, # numeric ranges, presence of negation keywords
constraint	# prohibitive statements, # permissive statements
strategy	# imperative sentences, presence of ordering keywords
eval	# distinct metric names, # comparison operators ($<, >, \leq$), # weighted terms
loop	presence of termination condition, # numbered steps, “NEVER STOP”
V (global)	total token count, type-token ratio, mean sentence length

4 Defining the equivalence relation formally

We define three levels of structural equivalence:

Coarse (type-signature equivalence):

$$p \sim_\sigma q \iff \sigma(p) = \sigma(q)$$

Programs with the same section types present, in any order. This partitions $\mathcal{T}$ into a finite number of classes.

Fine (feature equivalence):

$$p \sim_\phi q \iff \sigma(p) = \sigma(q) \wedge \|\phi(p) - \phi(q)\|_\infty < \varepsilon$$

for some tolerance $\varepsilon > 0$. This partitions programs into cells of a grid over $\mathbb{R}^d$.

Finest (tree-isomorphism equivalence):

$$p \sim_T q \iff T(p) \cong T(q)$$

Two programs are tree-isomorphic if their ASTs are identical up to the renaming of leaf content. The three relations form a strict lattice:

$$\sim_T \subsetneq \sim_\phi \subsetneq \sim_\sigma$$

5 The research pipeline as a function on the quotient

Once classes are fixed, we define the *outcome map*:

$$F : \mathcal{T}/\sim \longrightarrow \Delta(O)$$

where $\Delta(O)$ is the space of probability distributions over observable outcomes. The research question is: *at which level of this lattice does the autoresearch pipeline’s behaviour become constant within a class?*

